

LUA — Architecture Overview

Lily Upskilling Academy — skill-verification platform

Type: Skill-verification platform (assessments → AI/expert/certificate review → verified badges → public employer lookup), not an LMS.

Stack

Layer	Technology
Framework	TanStack Start (React 19 + Vite, file-based routing) — not Next.js
Routing	TanStack Router — routes in <code>src/routes/</code> , generated tree in <code>routeTree.gen.ts</code>
Data fetching	TanStack Query
Styling	Tailwind CSS v4 (CSS-first, no config file) + shadcn/ui (<code>src/components/ui/</code>)
Backend	Supabase (Postgres + Auth), accessed via TanStack Start server functions
AI	Lovable AI Gateway (MCQ generation, feedback) + Groq (conversational interview feature)
Build/host	Vite + Nitro (Cloudflare target), wrapped by <code>@lovable.dev/vite-tanstack-config</code>
Managed by	Lovable (lovable.dev) — syncs from a connected git branch

Request lifecycle

- 1. Routing:** file-based, under `src/routes/`. `_authenticated/` is a pathless layout whose `beforeLoad` calls `supabase.auth.getUser()` and redirects unauthenticated users to `/auth`. Every route under it inherits that guard and gets `{ user }` in context.
- 2. Rendering:** SSR on first load via TanStack Start + Nitro; client takes over via hydration. `__root.tsx` is the app shell (`<html>` / `<head>` / `<Scripts>`).
- 3. Layout split:** `DashboardChrome` picks between `AdminSidebar` (`/admin/*`) and `CandidateSidebar` (everything else) based on the URL path, and separately picks which notification bell to show the same way.

Server functions: the three-file pattern

Every server-backed feature is split into:

- `*-shared.ts` — plain types/constants, safe on client and server (e.g. `assessment-shared.ts`, `badges-shared.ts`).
- `*.functions.ts` — `createServerFn(...)` RPC definitions. Ships to the client as a thin stub; must never import secrets directly.

- `*.server.ts` — the real logic (service-role Supabase queries, AI gateway calls, `assertAdmin`). Loaded only from inside a `.functions.ts` handler.

This is enforced by ESLint (bans the `server-only` import pattern) so secrets can't leak into the client bundle by accident.

Auth

Three distinct Supabase clients, each with one job:

- `client.ts` — browser/SSR, RLS-scoped, publishable key.
- `client.server.ts` (`supabaseAdmin`) — service-role, bypasses RLS, server-only.
- `auth-middleware.ts` (`requireSupabaseAuth`) — validates the caller's JWT and injects an RLS-scoped client authenticated *as that user* into server function context.

Admin authorization is a Postgres RPC (`has_role()`, `SECURITY DEFINER`), checked via `assertAdmin()` against the user's own RLS-scoped client — not a client-side flag.

Data model (Postgres, RLS on every table)

- `app_role` enum (`candidate` | `employer` | `reviewer` | `admin`) + `user_roles` + `has_role()`.
- `skills` (catalog) / `user_skills` (a candidate's own portfolio — status, score, trust, active verifications).
- `questions` (bank) → `assessment_attempts` (graded server-side, `question_ids` fixed per attempt).
- `certificates` (AI-issued or uploaded, admin-approved, `is_public` gates the employer verify page).
- `badges` (`ai` | `certificate` | `institution` [placeholder], per `user_id` + `skill_id` + `type`) + `badge_appeals`.
- `admin_notifications` / `support_tickets` / `admin_audit_log` / `skill_requests` / `retake_requests`.
- "Fully Verified Bustler" is *computed*, not stored — true when every skill in a candidate's portfolio has ≥ 1 active verification (`isFullyVerifiedBustler()` in `skills-shared.ts`).

AI integration

Both features hit `ai.gateway.lovable.dev` with `LOVABLE_API_KEY`, model `openai/gpt-5.6-sol`, `response_format: json_object`, parsed defensively:

- `admin.server.ts::generateQuestionsWithAi` — draft question generation.
- `assessment.server.ts::evaluateWithAi` — post-attempt feedback; **never throws**, falls back to a templated summary.

Separately, an AI Interview feature (Groq-backed) provides a conversational verification path alongside the MCQ assessments.

Error handling (three layers)

1. `start.ts` — request-level middleware, rethrows intentional HTTP errors, logs+static-pages everything else.

2. `server.ts` — wraps the SSR fetch endpoint, detects h3-swallowed throws.
3. `error-capture.ts` — global `window.onerror` / `unhandledrejection` listener, gives `server.ts` a ~5s window to recover the real stack trace.

Client-side React errors go through `__root.tsx`'s `ErrorComponent`, forwarded to Lovable's `window.__lovableEvents` if present.

Known rough edges (as of this review)

- The dev toolchain (`vite@8.1.4` beta, `nitro@3.0.260603-beta`, pre-1.0 `@tanstack/react-start`) has real instability — hydration-mismatch races, a `process.env` define bug (worked around via a `window.process` polyfill in `__root.tsx`), dependency-optimizer reload glitches.
- Bustler SSO is fully specified (`docs/bustler-sso-integration.md`) but blocked on Bustler providing signing credentials — nothing to build on our side yet.
- Admin roles are currently just `admin` / `reviewer`, not the finer-grained "verification officer / support executive / content manager" split the product spec describes.